DistilBERT and CrisisMMD: Calibration and Privacy Budget Experiments

This report evaluates DistilBERT as a model for adaptive differential privacy (DPA-DP) in a simulated disaster scenario, following the earlier calibration work on MobileNetV3.

Without privacy noise, DistilBERT was calibrated on the Emotion dataset (learning rate 0.0001, at least one unfrozen transformer layer, 87-90% accuracy) and then on the text part of CrisisMMD 2.0 (67-69% accuracy, a lower ceiling because the tweets are noisier and the labels more ambiguous).

With DP-SGD enabled, a clipping norm sweep showed that accuracy is unchanged from 0.0001 to 1 and drops only below that, so 0.0001 was used. Four noise-multiplier profiles (one fixed and three DPA-DP) were then compared on a simulated crisis: 180 normal rounds, then 20 rounds with a mix of Emotion and CrisisMMD messages, all with a matched final privacy budget of about epsilon 14.2.

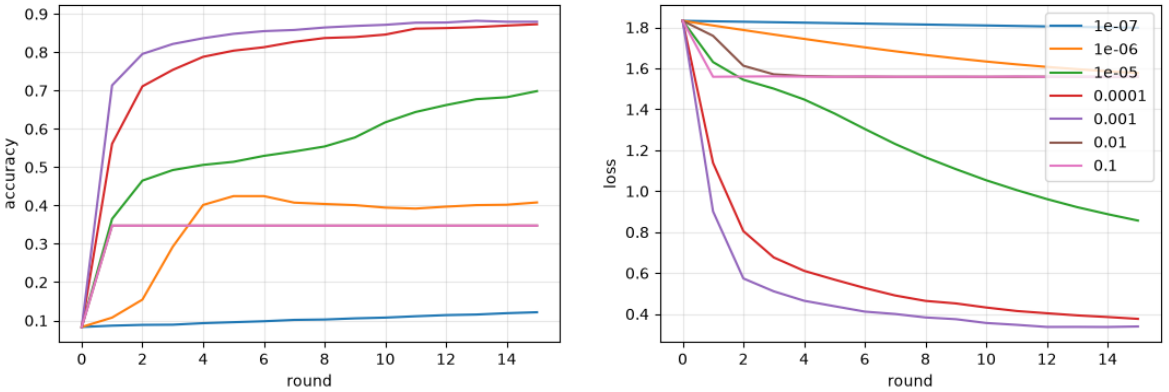
All DPA-DP profiles were slightly worse than the fixed baseline before the crisis but better after it. The profile with the largest phase difference (1.35 normal, 0.65 disaster) performed best, gaining about 1.6 points overall and 4.4 points on disaster messages over the fixed baseline.

DistilBERT performance by learning rate (August)

noise_multiplier: 0, tune_layers: 1, batch_size: 32

DistilBERT model on the Emotion dataset (16,000 training / 2,000 test texts) with privacy noise disabled. Accuracy and loss both improved steadily as the learning rate increased from 1e-7 to 0.001, but at 0.01 and above the model stopped learning, producing effectively random output. Learning rate 0.0001 was selected for the following experiments as it gives good results and far enough from the collapse value.

In [5]:
```
fig, (ax1, ax2) = pyplot.subplots(1, 2, figsize=(13, 4))
flwr_accuracy_sub(ax1, "../05_CrisisMMD/bert_clear_learning", "accuracy")
flwr_accuracy_sub(ax2, "../05_CrisisMMD/bert_clear_learning", "loss", key
```

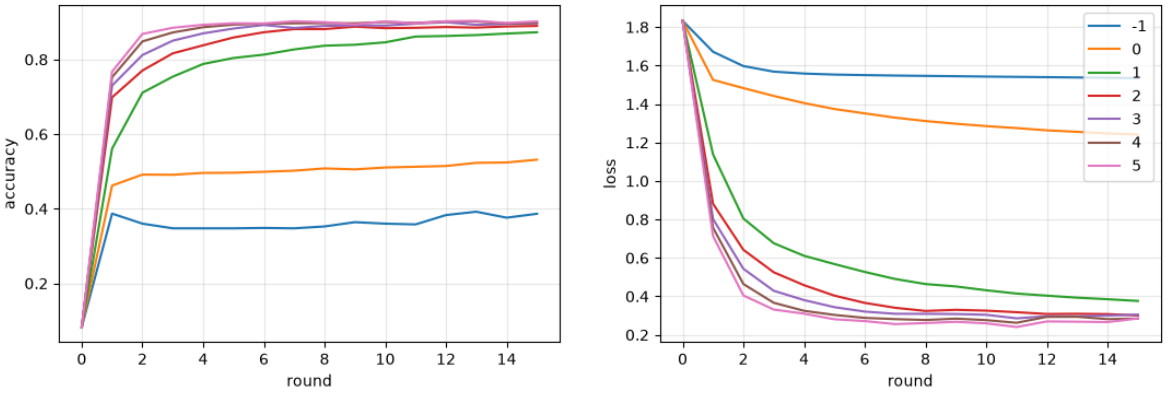


DistilBERT performance by frozen layers (August)

noise_multiplier: 0, learning_rate: 0.0001, batch_size: 32

With layers 2, 3 and above unfrozen, the model converges to 88-90% accuracy on the Emotion dataset. However, a fully frozen backbone with only the classifier head trainable, was unable to provide result better than 60%, indicating that at least one unfrozen backbone layer is necessary for DistilBERT performance.

In [6]:
```
fig, (ax1, ax2) = pyplot.subplots(1, 2, figsize=(13, 4))
flwr_accuracy_sub(ax1, "../05_CrisisMMD/bert_clear_layers", "accuracy")
flwr_accuracy_sub(ax2, "../05_CrisisMMD/bert_clear_layers", "loss", key="
```



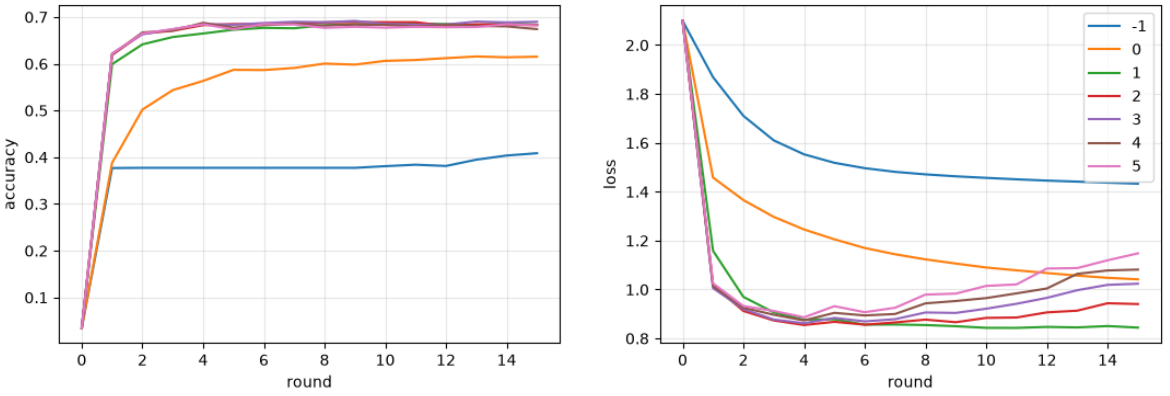
DistilBERT performance on CrisisMMD (August)

noise_multiplier: 0, learning_rate: 0.0001, batch_size: 32

In this test the Emotion dataset was replaced with the text modality of CrisisMMD 2.0, a state of the art dataset of social media messages and images from various disaster types. It defines 8 labels and is pre-split by its authors into 13,608 training and 2,237 test texts.

Configurations with layer 1 and above unfrozen converge to 67-69% accuracy. Models with more unfrozen layers show a higher rise in loss value during later training rounds.

In [7]:
```
fig, (ax1, ax2) = pyplot.subplots(1, 2, figsize=(13, 4))
flwr_accuracy_sub(ax1, "../05_CrisisMMD/crisismmd_layers", "accuracy")
flwr_accuracy_sub(ax2, "../05_CrisisMMD/crisismmd_layers", "loss", key="l
```



DistilBERT DPA-DP privacy budget allocation

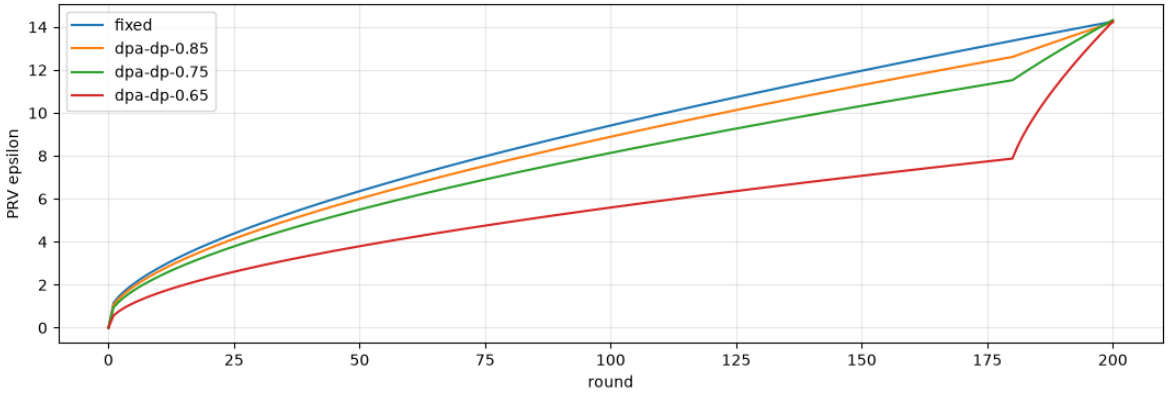
Epsilon budget consumption over 200 rounds (180 normal + 20 simulated disaster phase), for four noise-multiplier profiles tuned to reach a matched final budget of PRV epsilon around 14.2.

- fixed - uniform noise multiplier 1.0 across both phases
- dpa-dp-0.85 - low profile: 1.03 normal, 0.85 disaster
- dpa-dp-0.75 - medium profile: 1.08 normal, 0.75 disaster
- dpa-dp-0.65 - high profile: 1.35 normal, 0.65 disaster, the largest phase difference

In [8]:
```
rounds=200
noise_multiplier_switch = 180
batch_size = 32
n_samples = 1600 # emotion 16000 training set / 10 clients

data = {}
data["fixed"] = epsilon(1.00, 1.00, PRVAccountant())
data["dpa-dp-0.85"] = epsilon(1.03, 0.85, PRVAccountant())
data["dpa-dp-0.75"] = epsilon(1.08, 0.75, PRVAccountant())
data["dpa-dp-0.65"] = epsilon(1.35, 0.65, PRVAccountant())
print({k: a[-1] for k, a in data.items()})
plot_epsilon(data, "PRV epsilon")
```

```
{'fixed': 14.247314393214502, 'dpa-dp-0.85': 14.216532273266143, 'dpa-d
p-0.75': 14.322377944125195, 'dpa-dp-0.65': 14.255579900150469}
```



Text dataset for crisis simulation

Similarly to the MobileNetV3 experiment, we created a dataset containing normal messages for the normal stage (16,000 texts from the Emotion dataset). After round 180 (crisis onset), these were replaced with a mix of normal and disaster messages (8,000 Emotion + 8,000 CrisisMMD v2.0). The detailed labels and text counts are given in the table below. Accuracy was evaluated on the merged test splits of both datasets.

In [9]:
```
df = pandas.read_csv("dataset_size.csv")
display(df)
```

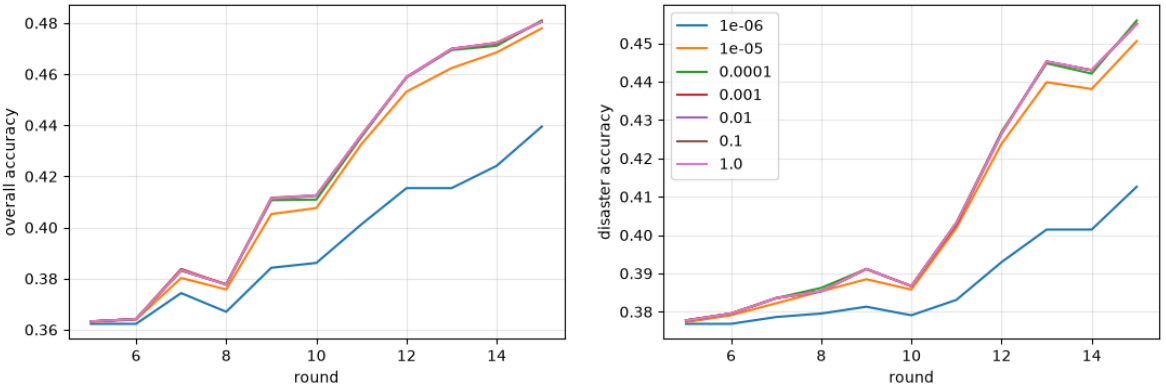
	label	Emotion	CrisisMMD	combined
0	sadness	4666	0	2333
1	joy	5362	0	2681
2	love	1304	0	652
3	anger	2159	0	1080
4	fear	1937	0	968
5	surprise	572	0	286
6	affected_individuals	0	384	226
7	infrastructure_and_utility_damage	0	1102	648
8	injured_or_dead_people	0	393	231
9	missing_or_found_people	0	29	17
10	not_humanitarian	0	3970	2334
11	other_relevant_information	0	4817	2832
12	rescue_volunteering_or_donation_effort	0	2867	1685
13	vehicle_damage	0	46	27
14	train	16000	13608	16000
15	test	2000	2237	4237

DistilBERT accuracy by clipping norm

tune_layers: 2, learning_rate: 0.001, batch_size: 32, noise_multiplier: 1

We ran 15 rounds on the combined dataset (8,000 Emotion + 8,000 CrisisMMD) with the fixed profile (sigma = 1) and different max_grad_norm values (the DP-SGD clipping norm). Accuracy decreases when the value falls below 0.0001, but it remains unchanged across a wide range from 0.0001 to 1. This contradicts the recommendation of the Google Research paper "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy", which advises choosing the smallest clipping norm (ClipSearch sweep).

In [10]:
```
fig, (ax1, ax2) = pyplot.subplots(1, 2, figsize=(13, 4))
flwr_accuracy_sub(ax1, "norm/norm10_all", "overall accuracy", 5)
flwr_accuracy_sub(ax2, "norm/norm10_sub", "disaster accuracy", 5, loc="up
```

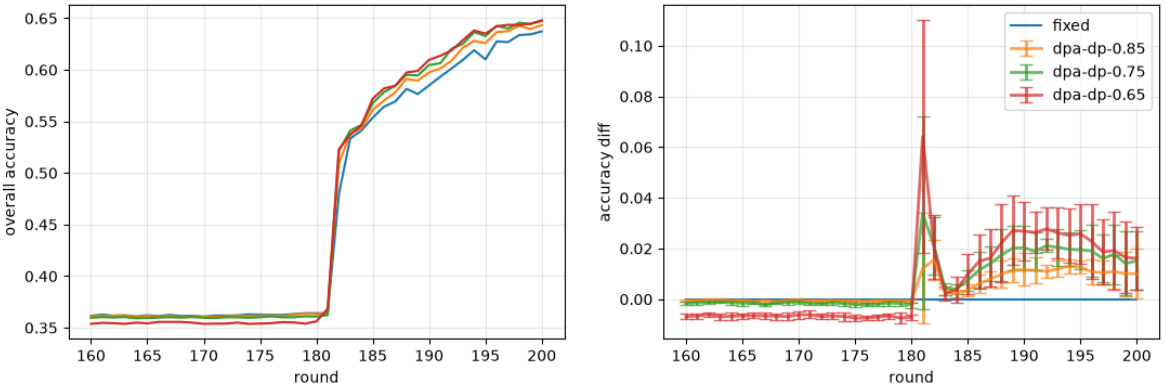


Accuracy between privacy profiles

tune_layers: 2, learning_rate: 0.001, batch_size: 32, max_grad_norm: 0.0001

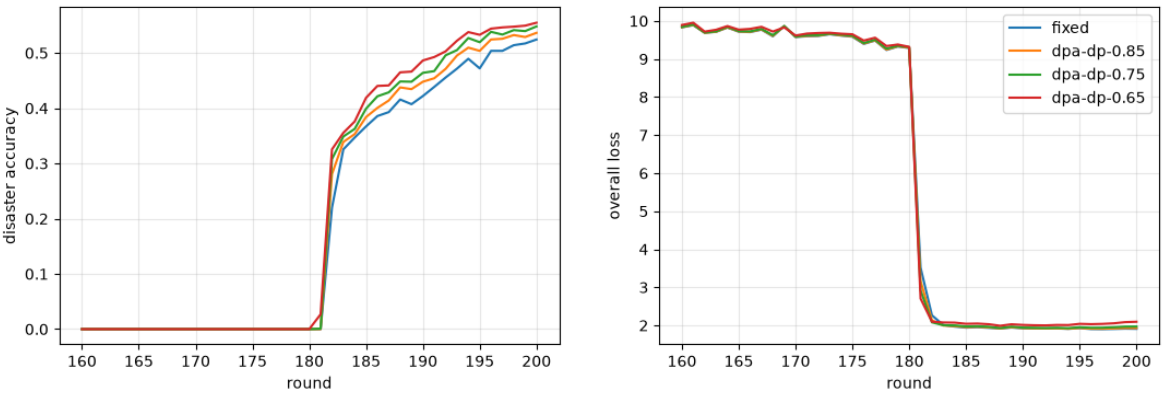
Two unfrozen layers plus the classifier were chosen as a middle ground between reasonable accuracy and a reduced number of parameters exposed to the privacy noise. During normal rounds, all DPA-DP profiles show slightly worse but comparable accuracy to the fixed baseline. After the simulated disaster onset at round 180, all profiles show better accuracy than the fixed baseline, with the high profile performing best.

In [11]:
```
fig, (ax1, ax2) = pyplot.subplots(1, 2, figsize=(13, 4))
flwr_accuracy_sub(ax1, "profiles/flwr_all", "overall accuracy", 160)
flwr_accuracy_error(ax2, "profiles/flwr6_all", 160)
```



<Figure size 1100x600 with 0 Axes>

In [12]:
```
fig, (ax1, ax2) = pyplot.subplots(1, 2, figsize=(13, 4))
flwr_accuracy_sub(ax1, "profiles/flwr_sub", "disaster accuracy", 160)
flwr_accuracy_sub(ax2, "profiles/flwr_all", "overall loss", 160, key="los
```



In [14]:
```
accuracy_summary("profiles/flwr6_all")
```

Out[14]:

	mean	deviation	diff	dev_diff
fixed	0.6112	0.0219	0.0000	0.0000
dpa-dp-0.85	0.6224	0.0201	0.0113	0.0053
dpa-dp-0.75	0.6294	0.0176	0.0182	0.0087
dpa-dp-0.65	0.6336	0.0147	0.0225	0.0120

In [15]:
```
accuracy_summary("profiles/flwr6_sub")
```

Out[15]:

	mean	deviation	diff	dev_diff
fixed	0.4783	0.0417	0.0000	0.0000
dpa-dp-0.85	0.5004	0.0382	0.0221	0.0098
dpa-dp-0.75	0.5149	0.0334	0.0366	0.0161
dpa-dp-0.65	0.5302	0.0274	0.0519	0.0224

In [16]:
```
table
```

Out[16]:

	All labels			Disaster labels		
	Event gain	**No-event**	**Advantage**	**Event gain**	**No-event**	**Advantage**
dpa-dp-0.85	+1.13 ± 0.35	-0.04 ± 0.03	+1.17 ± 0.38	+2.21 ± 0.62	+0.00 ± 0.03	+2.21 ± 0.62
dpa-dp-0.75	+1.82 ± 0.71	-0.20 ± 0.06	+2.02 ± 0.76	+3.66 ± 1.28	-0.08 ± 0.07	+3.74 ± 1.29
dpa-dp-0.65	+2.25 ± 1.06	-1.31 ± 0.19	+3.55 ± 1.20	+5.19 ± 1.98	-0.96 ± 0.27	+6.15 ± 2.20

In [ ]: